

UNIVERSIDAD DE VALPARAÍSO
Facultad de Ingeniería
Escuela de Ingeniería en Informática

Entradas y salidas digitales

Juego de memoria audiovisual
«Simón Dice» con Arduino UNO

Informe técnico · Implementación en simulación

Estudiante: Maximiliano Bernal
Asignatura: CIN-212 Hardware Digital
Docente: Jhon Pilataxi
Ayudante: Andrés González
Fecha: 28 de septiembre de 2026

1. Introducción y objetivos

El proyecto implementa un juego de memoria audiovisual con una placa Arduino UNO simulada en Wokwi. El sistema presenta una secuencia de luces y notas musicales que el jugador debe repetir mediante tres pulsadores. Cada nivel conserva la secuencia anterior y agrega un paso, aumentando progresivamente la dificultad.

La experiencia integra entradas y salidas digitales, resistencias limitadoras, temporización, generación pseudoaleatoria y filtrado de rebotes. La validación descrita corresponde a simulación; no se realizó montaje físico para este informe.

Objetivo general

Diseñar e implementar un juego de secuencias sobre Arduino UNO que permita capturar pulsaciones, comprobar respuestas y entregar señales visuales y sonoras de acierto o error.

Objetivos específicos

Configurar GPIO mediante `pinMode`, `digitalRead` y `digitalWrite`; emplear `INPUT_PULLUP` en los pulsadores; filtrar cambios con `millis`; proteger los LED mediante resistencias; organizar el programa en funciones; y documentar la simulación y el código en un repositorio.

Descripción funcional

Al iniciar se configura el hardware y se establece la semilla mediante `randomSeed(analogRead(A0))`. El primer nivel contiene un paso. Durante la presentación, cada LED se enciende 500 ms acompañado de su nota, seguido de 200 ms de pausa. Después se habilita la lectura del jugador. Una respuesta correcta completa activa una melodía de éxito; una respuesta incorrecta activa el aviso de error y limpia la partida.

Acceso al proyecto

Simulación:

<https://wokwi.com/projects/476374630777247745>

Repositorio:

<https://github.com/rulayy/simon-dice-cin212>

El repositorio permite conservar el código, el archivo de conexiones y la documentación. El trabajo se desarrolló de forma individual. La documentación y el circuito se almacenan junto al código para facilitar su revisión y reproducción.

2. Diseño de hardware y conexiones

Elemento	Pin	Conexión
Pulsadores rojo / verde / azul	D2 / D3 / D4	Contacto restante a GND; INPUT_PULLUP
LED rojo / verde / azul	D8 / D9 / D10	Ánodo al pin; cátodo a 220 Ω y GND
Buzzer pasivo	D11	Positivo al pin; negativo a GND
Entrada de semilla	A0	Sin conexión externa

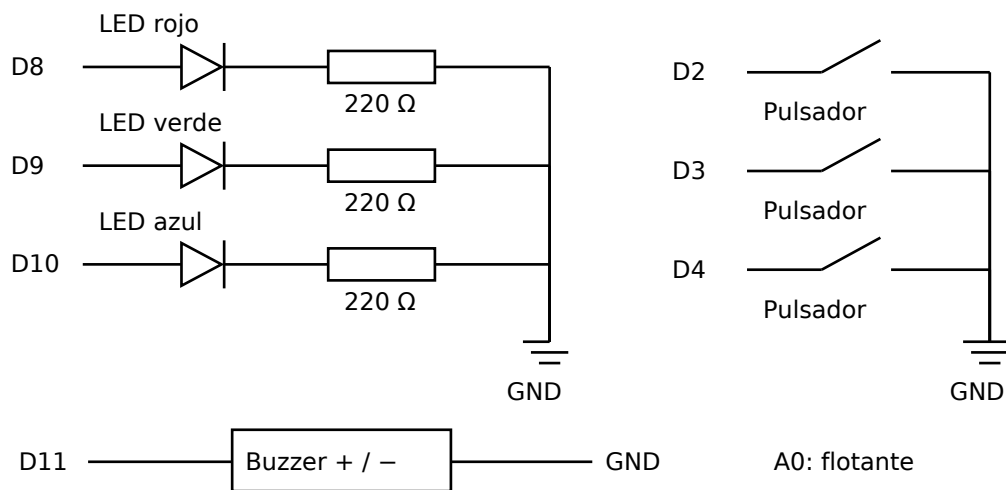


Figura 1. Esquema eléctrico funcional. Las entradas D2–D4 utilizan las resistencias pull-up internas; todos los GND son comunes.

Cálculo de resistencias

Por ley de Ohm, $R = (V_{\text{salida}} - V_f) / I$. Con 5 V y una caída de ejemplo $V_f = 2$ V, la resistencia de 220 Ω produce $I = (5 - 2) / 220 = 13,6$ mA. Su disipación es $P = I^2 R \approx 0,041$ W, inferior a 0,25 W. Con $V_f = 3$ V, la corriente estimada baja a 9,1 mA. Estos cálculos usan valores típicos: el montaje físico exige comprobar la ficha de cada LED y los límites por pin y agregados del microcontrolador. La especificación del UNO indica 20 mA por pin de E/S [1].

La conexión propuesta del buzzer supone un piezo pasivo de bajo consumo. Antes de emplear otro modelo físico se debe verificar su corriente y si requiere una etapa de accionamiento.

3. Lógica del software

El programa separa presentación y captura en dos estados: NUEVO_NIVEL y RESPUESTA. Las funciones de sonido, éxito, error y reinicio mantienen el loop breve. La secuencia se almacena en un arreglo de 100 posiciones; al completarse ese límite se inicia otra partida para evitar accesos fuera de memoria.

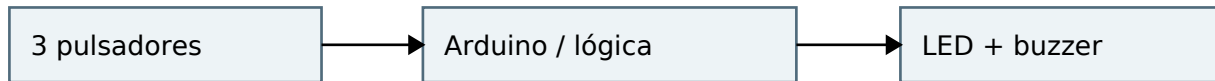


Figura 2. Bloques del sistema: captura, procesamiento y salida audiovisual.

Seudocódigo del flujo de control

1. Configurar pines e inicializar la semilla pseudoaleatoria.
2. Agregar un color aleatorio entre 0 y 2 a la secuencia.
3. Reproducir la secuencia completa con luces y sonidos.
4. Esperar liberación de botones y leer una pulsación estable.
5. Si es incorrecta: emitir error, limpiar secuencia y volver a 2.
6. Si es correcta: avanzar al siguiente paso esperado.
7. Si quedan pasos: volver a 4; si termina el nivel: emitir éxito y volver a 2.

Antirrebote y temporización

Cada cambio de lectura reinicia una marca de tiempo. Solo se actualiza el estado estable cuando transcurren 30 ms sin cambios, usando millis. Después de aceptar una entrada se exige liberar todos los botones antes de registrar otra. Esto evita que una pulsación sostenida cuente varias veces. El filtrado no usa delay; los retardos se reservan para presentación y señales, fases en las que no se aceptan respuestas.

Tipo	Uso y rango
byte	Colores 0-2; índices y longitud 0-100.
int	Entrada 0-2, -1 sin evento y -2 pulsación múltiple.
unsigned int	Frecuencias y duraciones breves.
unsigned long	Marcas de millis y diferencias de tiempo.
bool / enum	Espera de liberación y estado de ejecución.

4. Pruebas y resultados de simulación

Se ejecutó el juego en Wokwi y se alcanzó el nivel 5. La captura muestra un nivel completado y el inicio del nivel 5; no demuestra que este último haya sido completado. Posteriormente se ingresó una respuesta incorrecta y se observó el retorno al nivel 1, según el registro de la prueba del estudiante.

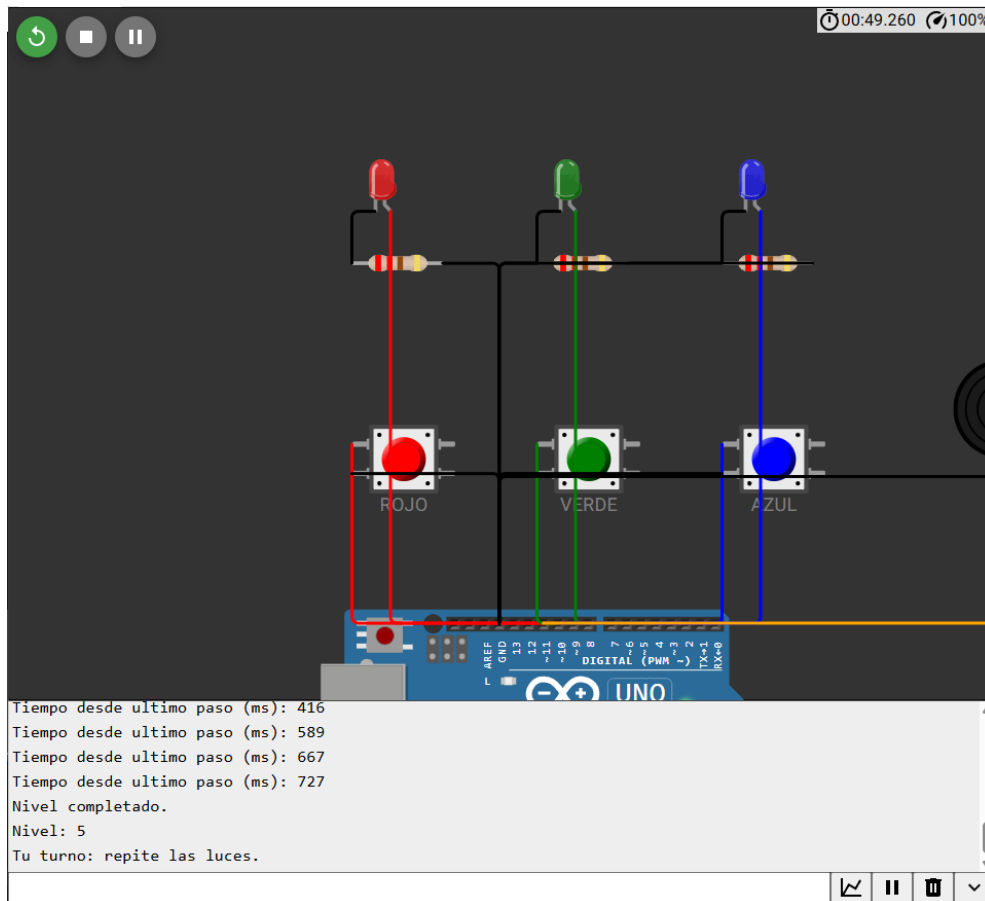


Figura 3. Captura de la simulación: circuito y monitor serial al alcanzar el nivel 5.

Prueba	Resultado y evidencia
Avance de nivel	Nivel 5 alcanzado; visible en la captura.
Respuesta incorrecta	Retorno al nivel 1 confirmado por el estudiante.
Intervalos entre entradas	416, 589, 667 y 727 ms visibles en el monitor serial.

Los cuatro intervalos visibles tienen un promedio de 599,75 ms. Son tiempos entre entradas aceptadas del jugador; no representan la latencia eléctrica del pulsador ni una medición aislada del antirrebote. No se obtuvieron mediciones físicas. La captura por sí sola tampoco verifica los tonos o el parpadeo de error.

5. Conclusiones y aprendizajes

La simulación permitió comprobar la progresión hasta el nivel 5 y el reinicio al nivel 1 después de una respuesta incorrecta. La organización por funciones relaciona cada requisito con una responsabilidad específica y facilita revisar las conexiones, la captura de entradas y las transiciones del juego.

El uso de INPUT_PULLUP define el nivel lógico de reposo y simplifica el cableado. El filtrado temporal y la exigencia de liberar los botones abordan dos problemas distintos: los rebotes mecánicos y las entradas repetidas por mantener un botón presionado. La protección con resistencias vincula el control lógico con los límites eléctricos de los componentes.

La principal limitación de esta validación es que se realizó en un simulador. Quedan por comprobar en hardware los niveles eléctricos, el comportamiento real de los pulsadores, el consumo y la señal acústica. Asimismo, una entrada analógica flotante no garantiza entropía suficiente en todas las ejecuciones, especialmente en simulación; la secuencia debe describirse como pseudoaleatoria.

Como continuación se propone probar pulsaciones sostenidas y múltiples, documentar el comportamiento con rebote activado y repetir la experiencia con una placa física. Los resultados disponibles no permiten afirmar que estos ensayos adicionales ya se hayan realizado en Wokwi.

6. Referencias

[1] Arduino, “UNO Rev3”. [En línea]. Disponible: <https://docs.arduino.cc/hardware/uno-rev3/>.

[2] Arduino, “Debounce on a Pushbutton”. [En línea]. Disponible: <https://docs.arduino.cc/built-in-examples/digital/Debounce/>.

[3] Wokwi, “wokwi-pushbutton Reference”. [En línea]. Disponible: <https://docs.wokwi.com/parts/wokwi-pushbutton>.

[4] Wokwi, “wokwi-buzzer Reference”. [En línea]. Disponible: <https://docs.wokwi.com/parts/wokwi-buzzer>.

[5] Universidad de Valparaíso, “CIN-212 Hardware Digital: Tarea 1, Entradas y Salidas Digitales”, enunciado de la asignatura, versión 1.0, 2026.